COMPANION HANDOUT

Build Your First Agent Harness

A five step, hands on methodology for turning a capable model into a system that does reliable work. Templates, configuration you can paste, and a self audit you can run on any agent setup you already have.

01	Create your system of record	30 minutes
02	Build your verification loop	2 to 3 hours
03	Get deliberate about tools	1 week
04	Read your failures	Ongoing habit
05	Ship one complete harness	1 weekend

THE FIVE COMPONENTS YOU ARE BUILDING

01	02	03	04	05
System of record	Tools	Feedback and verification	Guardrails and permissions	Observability and memory

Aishwarya Srinivasan
Co-founder, The Gen Academy

Harness Engineering
Video companion, Section 4

BEFORE YOU START

What this handout is, and how to work through it

The video gave you the concept and the anatomy. This is the build guide. Each of the five steps below has an objective, the exact actions to take, configuration you can copy, and a short checklist so you know when you are done. **Work the steps in order.** Step 5 depends on the four before it.

01	**Create your system of record**	Instruction files the agent reads before it works
02	**Build your verification loop**	Checks the agent runs on itself, automatically
03	**Get deliberate about tools**	Study a real harness, then extend it carefully
04	**Read your failures**	Turn failed runs into a prioritised backlog
05	**Ship one complete harness**	All five components around one real workflow
A	**The four failure modes**	What breaks, and the fix for each
B	**Harness self audit scorecard**	Grade any agent setup out of 15
C	**Reading list**	Primary sources, in the order to read them

The one idea underneath all five steps

A language model, on its own, is a pure function. Tokens in, tokens out. No network, no disk, no memory across calls, no ability to run anything or verify anything. Every useful behaviour you associate with AI comes from the layer wrapped around that function. That layer is the harness, and unlike the weights, it is entirely yours to design.

Prompt engineering	Optimises a single exchange. One instruction, one output.
Context engineering	Manages what the model can see at once. What gets retrieved, what fits in the window.
Harness engineering	Designs the execution environment for an agent that runs on its own: tools, verification, permissions, memory, logging.

Everything in this guide follows from one operating rule, which comes from Mitchell Hashimoto, co-founder of HashiCorp and creator of Terraform and Ghostty. He described it as the stage where his AI adoption changed character:

> **THE RULE THAT GENERATES YOUR WHOLE BACKLOG**
>
> "Anytime you find an agent makes a mistake, you take the time to engineer a solution such that the agent never makes that mistake again."
>
> Note the phrasing. Not *explain it better*. Engineer it so the mistake is no longer possible. A correction typed into chat dies with the session, because the model carries nothing between calls.

The five components you are building toward

01 System of record — *What does it need to know?*

Instruction files the agent reads before it starts work: project structure, commands, conventions, decisions your team already made. If knowledge lives in Slack threads or a wiki the agent cannot reach, it may as well not exist.

Looks like: `AGENTS.md`, `CLAUDE.md`, a `docs/` tree the root file points into.

02 Tools — *What can it actually do?*

The set of actions the agent is allowed to take, and the schema for each. Most setups fail at one of two extremes: everything is allowed, so behaviour is unpredictable, or almost nothing is, so the agent can talk but not act.

Looks like: shell access, file edit, test runner, browser driver, MCP servers into your real systems.

03 Feedback and verification — *How does it know it worked?*

Machinery that lets the agent check its own work without asking you. This is the component most people skip, and the one that separates a demo from something you can leave running.

Looks like: linter, type checker, real test suite, end to end run, a validation script for non code outputs.

04 Guardrails and permissions — *What must it never do?*

Explicit boundaries: which paths are read only, which commands need approval, where the agent stops and hands control back rather than guessing.

Looks like: allow, ask and deny rules, sandboxed execution, approval checkpoints before consequential actions.

05 Observability and memory — *What happened, and what carries over?*

A durable record of every tool call, decision and error, plus state that survives across sessions. When a run goes wrong, and it will, this record is the difference between diagnosis and guessing.

Looks like: session transcripts, structured logs, a progress file, a task list the agent updates as it goes.

01 Create your system of record

TIME: 30 MINUTES · COMPONENT: INSTRUCTION FILES · DO THIS TODAY

Write down what a competent new engineer would need on day one, put it where the agent will actually read it, and keep it under a page.

Why this is first

OpenAI's Codex team started an empty repository in late August 2025 and five months later had roughly one million lines of production code across about 1,500 merged pull requests, with the team growing from three engineers to seven. Their reported reason for slow early progress was not model capability. It was that **the environment was underspecified**. With no answer to a question like "which abstraction should I use here", the agent guessed, and guessed differently each time.

Where the file goes, per tool

TOOL	FILE IT READS	NOTES
OpenAI Codex	`AGENTS.md`	Repository root. Picked up automatically. Nested files in subdirectories are read for work in that subtree.
Cursor	`AGENTS.md`	Same open format.
Claude Code	`CLAUDE.md`	Reads `CLAUDE.md`, not `AGENTS.md`. Run `/init` to generate a starter. See the note below to keep one source of truth.
Others	`AGENTS.md`	Over 30 tools read the format, including GitHub Copilot, Gemini CLI, Zed, Aider, Jules, Windsurf, Devin, Warp.

> **CLAUDE CODE, ONE SOURCE OF TRUTH**
>
> Write everything in `AGENTS.md`, then create a `CLAUDE.md` containing `@AGENTS.md` plus any Claude specific overrides. The `@path` syntax imports the referenced file into context, up to four hops deep. On macOS and Linux a symlink works too: `ln -s AGENTS.md CLAUDE.md`.

The starter template

Fill in the sections, delete anything you cannot answer concretely. An honest half page beats a speculative three pages.

AGENTS.MD · REPOSITORY ROOT

```
# Project: <name>
<One sentence on what this is and who uses it.>

## Structure
- `src/<area>/`  <what lives here>
- `tests/`       <how tests are organised>
- `docs/`        Detailed guidance. Read the file for your area.

## Commands
- Install: `<exact command>`      - Lint:  `<exact command>`
- Run:     `<exact command>`      - Types: `<exact command>`
- Test:    `<exact command>`

## Conventions  # each line traces to a real past failure
1. <convention>   2. <convention>   3. <convention>
4. <convention>   5. <convention>

## Before you open a PR
- Run lint, types and tests. All three must pass.

## Do not
- Edit files under `<generated dir>`. Generated by `<command>`.
- Add a dependency without asking.
- Change or delete a test to make it pass.
```

Keep the root file small on purpose

OpenAI's team began with a comprehensive instruction manual and abandoned it. Long files crowded out the task, the code and the relevant docs. Their summary of the failure: **"when everything is important, nothing is."** What replaced it was a root `AGENTS.md` of roughly 100 lines acting as a table of contents, pointing into a structured `docs/` directory holding design docs, specs and execution plans. Detail still exists. It is just loaded when it is relevant instead of always.

Two mechanisms give you the same result without a large root file. **Nested files:** put an `AGENTS.md` inside a subdirectory, since agents read the nearest file in the tree and the closest one takes precedence, so frontend conventions live with the frontend. **Pointers:** keep the root file as an index naming `docs/` files by area, so detail is pulled in only when the agent is working there.

Then adopt the discipline

The file is not a document you write once. It is where corrections go to become permanent. Hashimoto reports that nearly every line in his project's `AGENTS.md` traces back to a specific bad agent behaviour, and that writing those lines "almost completely resolved them all". He pairs the file with purpose built scripts, for example to capture screenshots or run a filtered subset of tests, and documents them in the file so the agent knows they exist.

> **YOUR PERSONAL ESCALATION LADDER**
>
> **First time:** correct it in chat and move on. **Second time:** stop, and write it as a line in the instruction file. **Third time:** the instruction is not holding, so convert it into a check the agent cannot ignore, such as a lint rule, a test, or a pre-commit hook.

What a real convention line looks like

The difference between a file that works and one that gets ignored is specificity. A convention the agent can act on names the thing, the rule and the alternative. Compare:

TOO VAGUE TO ACT ON	SPECIFIC ENOUGH TO FOLLOW
Write clean, idiomatic code.	Use `async/await`. Do not add new `.then()` chains.
Follow our testing conventions.	Tests live beside the file they test as `<name>.test.ts`. Use the shared factory in `tests/factories.ts`, never inline fixtures.
Be careful with the database.	Migrations are append only. Never edit a file under `migrations/` that has already been applied.
Handle errors properly.	Throw typed errors from `src/errors.ts`. Never swallow an exception with a bare `catch` and a log line.

A useful test before you commit a line: could a competent engineer who has never seen this repository follow it without asking a follow up question? If not, the agent cannot either.

DONE WHEN

- [ ] An instruction file exists at your repository root, in the format your tool actually reads.
- [ ] It is one page or shorter, and every command in it has been run and works.
- [ ] Structure, run, test and your top five conventions are all covered.
- [ ] There is a "do not" section with at least two real entries.
- [ ] If you use more than one agent tool, they share a single source file via import or symlink.
- [ ] You have committed it, so it is version controlled alongside the code it describes.

02 Build your verification loop

TIME: **2 TO 3 HOURS** · COMPONENT: FEEDBACK AND VERIFICATION · HIGHEST LEVERAGE STEP

The goal in one sentence: **the agent should discover its own mistakes before you do.** Everything in this step exists to close that loop without a human in it.

The three tiers, cheapest first

TIER	CATCHES	RUNS IN	WIRE IT TO FIRE
Static checks Linter, formatter, type checker	Style drift, unused code, type errors, banned patterns	Under a second	After every file edit
Unit and integration tests	Broken logic, regressions, contract violations	Seconds to minutes	Before every commit, and on demand
End to end verification Real run, browser, API call	Anything that only fails when the pieces are assembled	Minutes	Before the agent declares a feature done

Add them in that order. Tier one catches the highest volume of small errors at the lowest cost, and it is the tier that most reliably changes agent behaviour, because the feedback arrives immediately after the edit that caused it.

Python: Ruff and pytest

PYPROJECT.TOML

```toml
[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
# E,F pyflakes+pycodestyle · I import order · B bugbear · UP pyupgrade
select = ["E", "F", "I", "B", "UP"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q --strict-markers"
```

THE THREE COMMANDS YOUR INSTRUCTION FILE MUST NAME

```
ruff check . --fix     # lint, autofixing what it safely can
ruff format .          # format
pytest -q              # tests
```

JavaScript and TypeScript

PACKAGE.JSON SCRIPTS · GIVE THE AGENT ONE VERB PER CHECK

```json
"scripts": {
  "lint":   "eslint . --fix",
  "format": "prettier --write .",
  "types":  "tsc --noEmit",
  "test":   "vitest run",
  "verify": "npm run lint && npm run types && npm run test"
}
```

That last line matters more than it looks. A single `verify` entry point gives the agent one command to run and gives you one command to put in your instruction file, your hook and your CI. When those three drift apart, the agent starts passing locally and failing in review.

Make the checks fire automatically

An instruction that says "run the tests after every change" is a request. A hook is a guarantee. In Claude Code, hooks are shell commands the tool runs at defined points in its lifecycle, which gives you deterministic behaviour instead of relying on the model to remember.

.CLAUDE/SETTINGS.JSON · FORMAT ON EVERY FILE WRITE

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
          }
        ]
      }
    ]
  }
}
```

The mechanics worth knowing, because they determine whether a hook merely reports or actually enforces:

- Claude Code passes event data as **JSON on stdin**. For an edit, `.tool_input.file_path` holds the file that changed, which is why the example pipes through `jq`.
- **Exit 0** means no objection. **Exit 2** blocks the action and sends whatever you wrote to stderr back to the model as feedback it can act on. Any other exit code surfaces as a non blocking error.
- Useful events beyond `PostToolUse`: `PreToolUse` to block a call before it happens, `Stop` to run a final gate when the agent thinks it is finished, and `SessionStart` to inject fresh context at the top of every session.
- Run `/hooks` inside a session to confirm your configuration was picked up.

> **THE PATTERN THAT TURNS A CHECK INTO A GUARDRAIL**
>
> A `Stop` hook that runs your `verify` command and exits 2 with the failure output when it fails. The agent cannot end its turn on a broken build, and it receives the exact error text rather than a vague instruction to try again.

If your agent does not write code

The principle is unchanged: define what correct output looks like, write something that checks it mechanically, and make that check part of every run. Verification does not require a test framework, only a deterministic answer to "is this right".

AGENT OUTPUT	VERIFICATION YOU CAN AUTOMATE
Structured data or JSON	Validate against a schema. Reject on any failure and return the validator error to the agent.
A report or summary	Assert required sections exist, every figure cited appears in the source, no placeholder text remains.
A database or API change	Run against a disposable copy first, then diff the result against the expected shape.
A document or slide deck	Render it, then check the artefact opens, has the expected page count, and contains no unfilled template fields.
Anything with a browser	Drive it end to end. Anthropic's long running agent work used a Puppeteer MCP server so agents could test features the way a user would rather than trusting a dev server that merely started.

THE NON NEGOTIABLE RULE

The agent may never edit or delete a test to make a run pass. Put this in your instruction file in those words. Anthropic states it directly in their harness guidance, because an agent optimising for a green result will otherwise take the shortest path to green.

DONE WHEN

- [ ] A linter or formatter runs automatically after file edits, without the agent choosing to run it.
- [ ] You have a single `verify` command that runs every check, and it is named in your instruction file.
- [ ] At least a handful of real tests exist. Tests that assert something meaningful, not that the import worked.
- [ ] A failing check produces output the agent can read and act on, not just a red exit code.
- [ ] Your instruction file forbids editing or deleting tests to make a run pass.
- [ ] You have watched the agent hit a failing check and fix itself without your input at least once.

03 Get deliberate about tools

TIME: **1 WEEK** · COMPONENTS: TOOLS, GUARDRAILS · STUDY FIRST, THEN EXTEND

Before you build anything custom, spend a week inside one seriously engineered harness and watch how it behaves. You will learn more from observing a good one under load than from any tutorial, including this one.

Part A: the observation week

Pick one, Claude Code or Codex, and use it on real work for a week, keeping a running note against these questions. This is how you build intuition for the decisions a harness designer actually has to make.

WATCH FOR	WHAT YOU ARE LEARNING
When does it stop and ask permission?	Where a team that has seen thousands of failures decided the blast radius justified a human.
Which parts of your instruction file does it visibly act on?	Which instructions survive contact with a real task, and which get ignored.
How does it recover after a failing test?	What a working feedback loop looks like from the inside.
What does it do before it starts editing?	The orientation sequence: reading files, checking state, forming a plan.
Where does it get lost or repeat itself?	Your first candidate list for what to fix in your own harness.

Part B: set your permissions on purpose

Every serious agent tool exposes permission settings, and almost nobody edits them. Setting them explicitly is the cheapest guardrail you will ever add. In Claude Code these live in `.claude/settings.json`, with **deny evaluated first, then ask, then allow**.

.CLAUDE/SETTINGS.JSON · PERMISSIONS

```
{
  "permissions": {
    "allow": [
      "Bash(npm run *)",
      "Bash(git status)",
      "Read(src/**)",
      "Edit(src/**)"
    ],
    "ask": [
      "Bash(git push *)",
      "WebFetch"
    ],
    "deny": [
      "Read(.env)",
      "Read(secrets/**)",
      "Bash(rm -rf *)"
    ]
  }
}
```

Rule syntax by tool: `Bash(...)` takes glob patterns with `*` at any position, `Read(...)` and `Edit(...)` use gitignore style paths, `WebFetch(domain:example.com)` scopes by domain, and `mcp__<server>__<tool>` addresses MCP tools, with `mcp__server__*` covering a whole server. Settings merge across user, project and local files. One caveat worth internalising: a pattern rule is a filter, not a sandbox. `Bash(rm -rf *)` in deny does not catch `rm -fr`. Pair deny rules with an isolated environment for anything genuinely destructive.

Part C: connect one real system through MCP

The Model Context Protocol is the standard interface between agents and external services. Connect exactly one system you use daily, then stop and evaluate before adding a second.

.MCP.JSON · PROJECT ROOT, SHARED WITH YOUR TEAM

```
{
  "mcpServers": {
    "local-db": {
      "command": "node",
      "args": ["./servers/db.js"],
      "env": { "DB_URL": "${DB_URL}" }
    },
    "remote-api": { "url": "https://api.example.com/mcp", "type": "http" }
  }
}
```

Variables expand as `${VAR}` or `${VAR:-default}` in `command`, `args`, `env`, `url` and `headers`, which is what lets you commit this file without committing a secret. A bare `$VAR` does not expand.

OR ADD ONE FROM THE CLI

```
# stdio: everything after -- is passed to the server untouched
claude mcp add --transport stdio local-db -- node ./servers/db.js

# remote: --transport http, or sse for services that only expose SSE
claude mcp add --transport http remote-api https://api.example.com/mcp
```

ASK THIS BEFORE EVERY SINGLE CONNECTION

What is the worst thing the agent could do with this access, and have I prevented it? Write the answer down. If your answer is "it would probably be fine", you have not thought about it yet.

Concretely: read only credential unless writes are the point, staging copy first, a `deny` rule for that server's destructive tools, and an `ask` rule for anything that leaves your machine.

The tool inventory, one row per capability

Fill this in for your setup. A tool you cannot justify in column two should be removed. A blank column three is an incident waiting to happen.

TOOL OR SERVER	WHY THE AGENT NEEDS IT	WORST CASE, AND THE CONTROL
example: shell	Run the test suite and build	Destructive command. Deny `rm -rf *`, run inside a container.

DONE WHEN

- [ ] You have used one production grade harness on real work for a week and taken notes on the five questions above.
- [ ] Your permission rules are written explicitly, including at least two deny rules for things you never want touched.
- [ ] Exactly one external system is connected, and you can state its blast radius in one sentence.
- [ ] Every tool in your inventory has a justification you would defend to a colleague.
- [ ] Credentials the agent uses are scoped to the minimum access the task needs.

04 Read your failures

TIME: **20 MINUTES PER FAILED RUN** · COMPONENT: OBSERVABILITY · THE COMPOUNDING HABIT

This is the least glamorous step and the highest leverage one. **Your harness roadmap is written in your failure logs.** You do not have to guess what to build next, because every failed run names its own missing component.

Find your logs

WHERE	WHAT IS THERE
Claude Code sessions	Transcripts are written to `~/.claude/projects/<project>/<session-id>.jsonl`, one JSON object per line. Use `claude --resume` to reopen a session and `/export` to save a readable copy. Treat the file format as internal, since it changes between versions.
Your own agent loop	Log every tool call with its arguments, the result, and the elapsed time. If you log nothing else, log this. Reconstructing a run without tool call history is guesswork.
The repository	Require a commit after each meaningful change with a descriptive message. This gives you a bisectable history of the agent's reasoning and a cheap way to revert to the last working state.
A progress file	Anthropic's harness guidance uses a progress file the agent updates as it works, so the next session can orient without re-deriving state. It doubles as a plain language audit trail.

The triage table

When a run goes wrong, read it from the top and find the first place the agent's behaviour diverged from what you expected. Then classify it. Almost every failure maps to exactly one missing component, and the fix follows from the classification.

WHAT YOU OBSERVE IN THE LOG	MISSING COMPONENT	THE FIX
It used the wrong command, package manager or API	System of record	Add one line to the instruction file. If it recurs, add a check that fails on the wrong usage.
It produced something plausible that was wrong, and nothing caught it	Verification	Add the check that would have caught it. Write the failing test first.
It thrashed, retried the same approach, or went in circles	Feedback loop	The signal it got was too weak. Return the actual error text, not a generic failure.
It touched something it had no business touching	Guardrails	Add a deny rule. Do not rely on an instruction to stay away from it.
It forgot a decision made earlier in the same run	Memory or context	Externalise the decision to a file the agent re-reads, rather than trusting it to stay in context.
It could not act, so it explained instead	Tools	The capability is genuinely missing. Add the tool, scoped narrowly.
You cannot tell what happened at all	Observability	Fix this before anything else. You cannot debug what you cannot see.

> **COUNTER INTUITIVE, AND LOAD BEARING**
>
> Do not scrub failed actions and error traces out of the agent's context on the next attempt. Manus argues that leaving errors in is what lets a model update its own priors and stop repeating a mistake, and calls error recovery one of the clearest indicators of genuinely agentic behaviour. Hiding the failure removes the only evidence the model has.

The failure log template

Keep one file, append one entry per failure. After ten entries the pattern in your own setup will be obvious, and it will not be the pattern you expected.

FAILURES.MD

```
## 2026-08-22 · Agent rewrote the migration instead of adding one
Component:  System of record
Evidence:   Session abc123, turn 14. No convention documented for migrations.
Fix:        AGENTS.md line: "Migrations are append only. Never edit an applied file."
Escalation: Added a pre-commit check rejecting edits to applied migrations.
Recurred:   No
```

DONE WHEN

- [ ] You know where your agent's logs live and have read one failed run start to finish.
- [ ] A `failures.md` file exists with at least three real entries.
- [ ] Each entry names one component and one concrete fix that has been applied.
- [ ] At least one entry has escalated from an instruction to an automated check.
- [ ] You can point at a mistake the agent used to make and no longer does.

05 Ship one complete harness

TIME: **ONE WEEKEND** · ALL FIVE COMPONENTS · THE GRADUATION PROJECT

Pick one repetitive, well defined task you do every week and build the entire loop around it. One workflow, end to end, running reliably without you watching. Doing this once teaches you more than reading about it ten times.

Choose the workflow

A good candidate scores yes on all five. If any answer is no, pick something else, because you will end up debugging the task rather than the harness.

	CRITERION	WHY IT MATTERS
1	You do it at least weekly	You will get enough runs to actually iterate.
2	You can define "correct" precisely	No verification is possible without this. It is the hard requirement.
3	Failure is cheap and reversible	You want to be able to let it run unsupervised.
4	It needs 3 to 10 steps, not 50	Small enough to finish this weekend, large enough to need a harness.
5	It touches at most two external systems	Keeps the blast radius and the debugging surface small.

Workable examples: triage new issues and label them by area. Turn a weekly data export into a formatted report. Update dependencies, run the suite, and open a PR only if it stays green. Convert design tokens into code and verify the build. Review inbound support tickets against a rubric and draft replies for approval.

Build in this order

The order matters. Each layer gives you the signal you need to build the next one well.

1 Write the spec before the harness

One file describing the goal, the inputs, the exact definition of a correct output, and the boundaries. Anthropic's approach expands a short prompt into an explicit list of requirements the agent works against, stored in a structured format with a pass or fail flag per item, so completion is a fact rather than a judgement call.

2 Write the verifier second, before any agent runs

A script that takes an output and returns pass or fail with a readable reason. Write it before the agent so you are forced to define correctness precisely. If you cannot write this, you have chosen the wrong workflow.

3 Give it the minimum tools

Start with fewer tools than you think it needs and add only when a failed run proves the gap. Every tool you add is surface area for both mistakes and debugging.

4 Add the initialisation step

Every run starts the same way: confirm the working directory, read the progress file and recent history, pick the highest priority incomplete item, start the environment, run a quick sanity check. Anthropic treats this orientation sequence as a distinct phase, and it is what stops an agent from acting on stale assumptions.

5 Set permissions, then log everything

Deny rules for anything destructive, an approval checkpoint at the last irreversible step, and a full record of every tool call. Only now let it run unsupervised.

6 Constrain scope per run

One unit of work per session, committed before the next begins. Anthropic's guidance is explicit that constraining each session to a single feature outperforms attempting the whole thing at once, and per step commits give you a working state to fall back to.

Two techniques worth stealing

Recitation. Manus has the agent maintain a `todo.md` and rewrite it as work proceeds. Restating the objective keeps it in recent context rather than buried behind fifty tool calls, which measurably reduces drift on long tasks. It is a cheap trick with an outsized effect.

The filesystem as memory. Rather than compressing a large observation into the context window and losing it, write it to a file and keep the path. Manus describes the filesystem as unlimited in size and persistent by nature, which makes it a better store than a context window for anything the agent might need to revisit.

Mask, do not remove. If you build your own loop, resist adding and removing tool definitions between turns. Keep the schema stable and constrain which actions are selectable instead. Removing a tool that earlier context already referenced invalidates cached prefixes and can produce schema violations when the model refers back to something that no longer exists.

> **SIMPLER, NOT BIGGER**
>
> The teams who have iterated longest converge on simpler harnesses. Manus rebuilt their agent framework four times, and their account of it reads as deliberate subtraction, not accumulation. Be as willing to delete from your harness as to add to it, and treat a growing instruction file as a warning sign rather than progress.

DEFINITION OF DONE

- [ ] The workflow runs end to end without you intervening, at least three times in a row.
- [ ] A spec file defines correct output, and a verifier checks it mechanically.
- [ ] The agent detects and recovers from at least one class of failure on its own.
- [ ] Permissions are explicit, and there is a human checkpoint before anything irreversible.
- [ ] You can reconstruct any run from its logs without guessing.
- [ ] You have removed at least one thing you originally thought was necessary.

APPENDIX A

The four failure modes

These are predictable. Knowing them in advance saves weeks, because each one feels reasonable at the time you commit it.

01 Patching failures in chat instead of in the environment

The agent uses the wrong package manager. You correct it in conversation. It apologises, complies, and three sessions later does exactly the same thing. Of course it does. The model carries nothing between sessions, so a chat correction has nowhere to live.

Fix: the correction has to land somewhere durable. A line in the instruction file, or better, a check that fails loudly when the rule is broken. A check the agent cannot ignore beats an instruction it might.

02 The thousand line instruction file

Six months in, every convention, historical decision and edge case has been appended to one enormous file, and the team is confused that the agent ignores half of it. Instructions compete for attention, and attention over a long context is not uniform. Two hundred instructions do not get followed two hundred times as well as one.

Fix: keep the root file under a page. Split detail into per area files the agent loads only when working there. Treat instruction budget as the constrained resource it is. This is exactly the correction OpenAI's team made when they replaced their instruction manual with a table of contents root file pointing into a structured `docs/` tree.

03 Skipping verification because the model is good now

This has the logic backwards. A strong model failing is more dangerous than a weak one failing, because a weak model's mistakes are obvious and a strong model's are camouflaged. It produces code that reads correctly, compiles, passes review, and is subtly wrong in a way that surfaces three weeks later in production.

Fix: your test suite does not care how confident the output looked. That is the entire point of having one.

04 Treating the harness as one time setup

You write the instruction file, wire up the checks, and consider it finished. But a harness is a living system. Every failure is feedback and every fix is a small permanent improvement, which means the harness should change roughly as often as your product does.

Fix: maintain it the way you maintain your product. Schedule a monthly pass where you delete rules that no longer earn their place. OpenAI's team went further and ran recurring background agents to detect stale documentation and drift from stated principles, rather than relying on humans to notice.

APPENDIX B

Harness self audit

Score any agent setup, yours or one you inherited, from 0 to 3 on each component. Fifteen is the ceiling and most working setups start between 3 and 6. The useful signal is not the total but **which component scores lowest**. Fix that one next.

COMPONENT	0 · ABSENT	1 · AD HOC	2 · DELIBERATE	3 · SELF CORRECTING
System of record	No instruction file. Context is re-explained every session.	A file exists but is stale, or written once and never revised.	Under a page, accurate, commands verified, detail split by area.	Lines trace to real failures. Stale entries removed on a schedule.
Tools	Chat only, or unrestricted access to everything.	Whatever the default was. Never reviewed.	A deliberate set. You can justify each one.	Scoped credentials, documented blast radius, unused tools removed.
Verification	You are the only check.	Tests exist but the agent does not reliably run them.	Checks fire automatically and the agent acts on the results.	Cannot finish on a failing check. Failures return actionable output.
Guardrails	Defaults, unexamined. Or all prompts bypassed.	Some approvals, chosen by convenience rather than risk.	Explicit allow, ask and deny rules. Sensitive paths denied.	Least privilege, isolated execution, checkpoints before irreversible acts.
Observability	No record. Failures are unreconstructable.	Logs exist somewhere. You have never opened them.	You read failed runs and act on what you find.	A failure log drives your backlog. State persists across sessions.

READING YOUR SCORE

`0 to 4`	You are a harness consumer using someone else's defaults. Step 1 and Step 2 will move you further than anything else you could do this month.
`5 to 9`	A real harness exists. Find your lowest scoring component and take it to a 2 before touching anything else.
`10 to 13`	Production grade. The remaining gains are in Step 4: read failures systematically and let the log set your priorities.
`14 to 15`	Now practise subtraction. Ask which rules, tools and checks no longer earn their place, and delete them.

RE-RUN THIS QUARTERLY

Scores drift downward on their own. Instruction files rot, tools accumulate, and checks disabled during a deadline never get re-enabled. Keep the previous score to compare against.

APPENDIX C

Reading list

Read the first four in order. They are the primary sources the whole discipline is currently built on, and each one is a practitioner account rather than commentary.

Start here

My AI Adoption Journey

READ FIRST

Mitchell Hashimoto. Six stages from AI sceptic to running agents continuously. Stage five is "engineer the harness", and the rule it states is the one that generates your entire backlog. Short, concrete, no theory.

`mitchellh.com/writing/my-ai-adoption-journey`

Harness engineering: leveraging Codex in an agent first world

READ SECOND

OpenAI. The five month project: roughly one million lines and about 1,500 merged PRs, with every line generated by Codex rather than hand written. The most detailed public account of what a harness looks like at scale: why their monolithic instruction file failed, how they made the application legible to the agent, what they did about entropy, and why blocking merge gates stopped working at high throughput.

`openai.com/index/harness-engineering/`

Effective harnesses for long running agents

READ THIRD

Anthropic Engineering. The most directly actionable of the four. A separate initialiser prompt, an explicit feature list with pass flags, one feature per session, commit after every change, end to end verification through a browser, and a fixed orientation sequence at the start of each run. Steps 2, 4 and 5 of this handout draw heavily on it.

`anthropic.com/engineering/effective-harnesses-for-long-running-agents`

Context Engineering for AI Agents: Lessons from Building Manus

READ FOURTH

Yichao Ji, Manus. The principles that survived four framework rebuilds: design around the KV cache, mask rather than remove tools, use the filesystem as unlimited context, manipulate attention through recitation, keep failed actions in context, and avoid few shot ruts. The most useful piece if you are building your own loop rather than extending an existing tool.

`manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus`

Specifications and documentation

AGENTS.md

SPEC

The open instruction file format, used by more than 60,000 open source projects and read by over 30 tools. Covers recommended sections, how nested files resolve in a monorepo, and a worked example.

`agents.md`

Model Context Protocol

SPEC

The standard interface between agents and external systems. Read the concepts pages before wiring anything up, then browse the reference servers to see how tools are scoped in practice.

`modelcontextprotocol.io`

Claude Code documentation: hooks, permissions, memory, MCP

DOCS

The four pages behind most of the configuration in this handout. Hooks for deterministic checks, permissions for guardrails, memory for how `CLAUDE.md` and imports resolve, MCP for connecting systems. The hooks reference in particular is worth reading in full once.

`code.claude.com/docs/en/hooks · /permissions · /memory · /mcp`

OpenAI Codex documentation

DOCS

How Codex discovers `AGENTS.md`, what its sandbox permits, and how approval modes work. Read alongside the harness engineering post above, which shows the same features under real load.

`developers.openai.com/codex`

Tooling for your verification loop

Ruff

PYTHON

Linter and formatter fast enough to run after every single file edit, which is what makes it work well as an agent feedback loop rather than a pre-commit afterthought.

`docs.astral.sh/ruff`

pre-commit

ANY LANGUAGE

The escalation target when an instruction is not holding. A rule enforced at commit time cannot be talked around, and it applies to human contributors too.

`pre-commit.com`

Playwright

END TO END

For the third verification tier. Lets an agent exercise a real interface and observe real failures instead of trusting that a dev server started successfully.

`playwright.dev`

> **ONE LAST THING**
>
> In 2026 the model is increasingly the commodity. Roughly the same frontier intelligence is available to everyone, which means what you compete on is the environment you build around it. You do not get a better agent by finding a better model. You get a better agent by building it a better place to work.
>
> Start with Step 1. It takes thirty minutes, and everything else compounds on top of it.